

快速沃尔什变换

简介

沃尔什变换（Walsh Transform）是在频谱分析上作为离散傅立叶变换的替代方案的一种方法。——[维基百科](#)

其实这个变换在信号处理中应用很广泛，FFT 是 double 类型的，但是 Walsh 把信号在不同震荡频率方波下拆解，因此所有的系数都是绝对值大小相同的整数，这使得不需要作浮点数的乘法运算，提高了运算速度。

所以，FWT 和 FFT 的核心思想应该是相同的，都是对数组的变换。我们记对数组 进行快速沃尔什变换后得到的结果为 。

那么 FWT 核心思想就是：

我们需要一个新序列 ，由序列 和序列 经过某运算规则得到，即；

我们先正向得到序列 ，再根据 在 的时间复杂度内求出，其中 是序列对应位置相乘；

然后逆向运算得到原序列 。时间复杂度为 。

在算法竞赛中，FWT 是用于解决对下标进行位运算卷积问题的方法。

公式：

（其中 是二元位运算中的某一种）

下面我们举 （按位或）、（按位与）和 （按位异或）为例。

FWT 的运算

或运算

如果有 ，那么 的二进制位为 的位置和 的二进制位为 的位置肯定是 的二进制位为 的位置的子集。

现在要得到 ，我们就要构造这个 FWT 的规则。

我们按照定义，显然可以构造 ，来表示 满足二进制中 为 的子集。

那么有：

那么我们接下来看 怎么求。

首先肯定不能枚举了，复杂度为 。既然不能整体枚举，我们就考虑分治。

我们把整个区间二分，其实二分区间之后，下标写成二进制形式是有规律可循的。

我们令 表示 的前一半，表示区间的后一半，那么 就是 A 下标最大值的最高位为 ，他的子集就是他本身的子集（因为最高位为 了），但是 的最高位是 ，他满足条件的子集不仅仅是他本身，还包最高位为 的子集，即

其中 merge 表示像字符串拼接一样把两个数组拼起来，就是普通加法，表示对应二进制位相加。

这样我们就通过二分能在 $\log n$ 的时间复杂度内完成拼接，每次拼接的时候要完成一次运算，也就是说在 $\log^2 n$ 的时间复杂度得到了。

接下来就是反演了，其实反演是很简单的，既然知道了 f 的本身的子集是他自己 (f) ， g 的子集是 f ，那就很简单的得出反演的递推式了：

下面我们给出代码实现。容易发现顺变换和逆变换可以合并为一个函数，顺变换时 $l = 0$ ，逆变换时 $l = 1$ 。

▼ 实现

```
1 void Or(ll *a, ll type) { // 迭代实现，常数更小
2   for (ll x = 2; x <= n; x <= 1) {
3     ll k = x >> 1;
4     for (ll i = 0; i < n; i += x) {
5       for (ll j = 0; j < k; j++) {
6         (a[i + j + k] += a[i + j] * type) %= P;
7       }
8     }
9   }
10 }
```

与运算

与运算类比或运算可以得到类似结论

下面我们给出代码实现。顺变换时 $l = 0$ ，逆变换时 $l = 1$ 。

▼ 实现

```
1 void And(ll *a, ll type) {
2   for (ll x = 2; x <= n; x <= 1) {
3     ll k = x >> 1;
4     for (ll i = 0; i < n; i += x) {
5       for (ll j = 0; j < k; j++) {
6         (a[i + j] += a[i + j + k] * type) %= P;
7       }
8     }
9   }
10 }
```

异或运算

异或的卷积是基于如下原理：

若我们令 f 表示 x 中 1 数量的奇偶性，即 $f(x) = x \bmod 2$ ，那么容易有 $f(x \oplus y) = f(x) \oplus f(y)$ 。

对于 f 的运算其实也很好得到。

设 g 。我们来证一下 g 的正确性：

来看看怎么快速计算 g 的值，依旧是分治：

对于 在当前位为 的子数列，进行 运算时发现它和 计算或和 计算结果都不会变（因为 ），所以 中的 。

对于 在当前位为 的子数列，进行 运算时发现它和 计算结果是 ，和 计算结果是 （因为 ）。

综上，有：

也就是：

逆变换易得：

给出代码，顺变换时，逆变换时 。

▼ 实现

```
1 void Xor(ll *a, ll type) {
2     for (ll x = 2; x <= n; x <= 1) {
3         ll k = x >> 1;
4         for (ll i = 0; i < n; i += x) {
5             for (ll j = 0; j < k; j++) {
6                 (a[i + j] += a[i + j + k]) %= P;
7                 (a[i + j + k] = a[i + j] - a[i + j + k] * 2) %= P;
8                 (a[i + j] *= type) %= P;
9                 (a[i + j + k] *= type) %= P;
10            }
11        }
12    }
13 }
```

同或运算

类比异或运算给出公式：

（表示 为，表示 为）

另一个角度的 FWT

我们设 是 对 的贡献系数。我们可以重新描述 FWT 变换的过程：

因为有：

所以我们可以通过简单的证明得到：。其中 是任意一种位运算。

同时，函数还有一个重要的性质，它可以按位处理。

举个例子，我们变换的时候：

这么做是比较劣的，我们将其拆分：

考虑前面的式子和后面的式子 的区别，发现只有最高位不同。

所以我们将 去除最高位的值为 ，并记 为 的最高位。有：

如果 ，则有：

则有：

也就是说，我们只需要：

四个数就可以完成变换了。我们称这个矩阵为位矩阵。

如果我们要进行逆变换，则需要上面的位矩阵的逆矩阵。

若逆矩阵为 ，可以通过类似操作得到原数：

逆矩阵不一定存在，比如如果有一排 或者一列 那么这个矩阵就没有逆，我们在构造时需要格外小心。

按位或

我们可以构造：

这样满足 。我们发现，这和我们前面推出的 一模一样！同理，下面也是一个满足这个条件的矩阵，但我们一般使用上面这个：

虽然下面这个矩阵也满足 ，但这个矩阵存在一排 ，不存在逆，所以不合法：

如果我们要进行逆变换，则需要对矩阵求逆，以 **最上面** 这个矩阵为例，得：

然后按照顺变换的方法，把逆变换矩阵代入即可。

按位与

我们可以构造：

这样满足 。

逆矩阵：

按位异或

我们可以构造：

这样满足 。

逆矩阵：

FWT 是线性变换

FWT 是线性变换。也就是说，它满足：

以及：

K 维 FWT

其实位运算的本质是对一个 K 维 向量的运算。或运算就是每一维取 $\vee$ 。且运算就是每一维取 $\wedge$ 。异或运算则是每一维对应相加再 $\bmod 2$ 。

位运算有个特点：向量的每一位都是独立的。

我们把 $\bmod 2$ 扩展到 $\bmod p$ 也就是扩展到 p 进制，看看会得到什么？

max 运算

我们将 $\bmod 2$ 运算拓展到 p 进制，定义 bit_i 表示按位取 i ，有：

若 $\text{bit}_i = i$ ，那么上式又是：

也就是说，每一行的 bit_i 必定只能在 i 的前面，如果在后面则不合法了。手玩一下可以发现一组合法构造：

求逆可得：

min 运算

我们将 $\bmod 2$ 运算拓展到 p 进制，定义 bit_i 表示按位取 i ，有：

若 $\text{bit}_i = i$ ，那么上式又是：

也就是说，每一行的 a_i 必定只能在 a_{i+1} 的后面，如果在前面则不合法了。手玩一下可以发现一组合法构造：

求逆可得：

前两者用得比较少，用得比较多的是：

不进位加法

我们将 $\oplus$ 运算拓展到 p 进制，定义 $\oplus$ 表示按位相加再 $\% p$ ，有：

我们构造 A ，就可以满足要求了：

但是每一行都一样矩阵也没有逆，所以我们可以构造 B 即可。

有下面这个矩阵：

此即为 [范德蒙德矩阵](#)，求逆可得：

如果我们题目给出的模数是存在单位根的，我们就可以简单实现。

但是 **单位根在模意义下可能不存在**，所以我们考虑扩域，就是人为地定义一个 ω ，满足 $\omega^p = 1$ ，然后直接把 ω 代入计算，这样每个数都是一个关于 ω 的 p 次多项式。我们只需要在 $\mathbb{Q}(\omega)$ 下计算即可。那么矩阵可以这么表示：

但是这么做可能会存在零因子，也就是 **一个数有多种表示方法**，我们无法确定一个数的真实值。

我们考虑不了，我们 p 分圆多项式，他满足 $\Phi_p(x)$ 的阶为 p ，且在 $\mathbb{Q}$ 上不可约。所以我们定义上面的计算是在 $\mathbb{Q}(\omega)$ 下进行即可。

还有一个问题是，常数大（因为 $\Phi_p(x)$ 本身就是一个多项式）。但是因为，我们只需要在计算时，最后再 $\% p$ 即可。

例题

▼ [「CF 1103E」Radix sum](#)

给定一个长度为 n 的序列 a ，对于每一个 i ，求满足下列条件的整数序列 b 的方案数，对 10^9+7 取模：

- $b_i \geq 0$ ；
- $a_i = b_i \oplus b_{i+1} \oplus \dots \oplus b_n$ ，这里的加法定义为十进制不进位加法。

► 题解

▼ [「CF103329F」【XXII OpenCup, Grand Prix of XiAn】The Struggle](#)

给出一个椭圆，其中所有整点的坐标均在 $[-10^9, 10^9]$ 之间。求 $\sum_{(x,y) \in E} x^2 + y^2$ 的值。

参考资料

- [桃酱的算法笔记](#)
 - [ZnPdCo 的博客](#)
-

本页面最近更新：2024/6/22 15:45:13, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Xeonacid](#), [ZnPdCo](#), [BinDir0](#), [Enter-tainer](#), [lr1d](#), [lxlonlyn](#), [nocriz](#), [nocrizwang](#), [Tiphereth-A](#), [TrisolarisHD](#), [xyf007](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用